

«Информатика и системы управления» (ИУ)

«Информационная безопасность» (ИУ8)

по дисциплине «Введение в искусственный интеллект»

## Вариант 2

(Подпись, дата)

(Подпись, дата)

Москва, 2026 г.

## СОДЕРЖАНИЕ

|     |                                                          |    |
|-----|----------------------------------------------------------|----|
| 1   | Цель работы . . . . .                                    | 2  |
| 2   | Постановка задачи . . . . .                              | 3  |
| 3   | Теоретические сведения . . . . .                         | 4  |
| 4   | Таблица истинности . . . . .                             | 5  |
| 5   | Программная реализация . . . . .                         | 5  |
| 6   | Результаты обучения на полной выборке . . . . .          | 6  |
| 6.1 | Функция активации №1 . . . . .                           | 6  |
| 6.2 | Функция активации №3 . . . . .                           | 7  |
| 6.3 | Итоговая проверка на полной таблице истинности . . . . . | 9  |
| 7   | Поиск минимального набора обучающих векторов . . . . .   | 10 |
| 7.1 | Функция активации №1 . . . . .                           | 10 |
| 7.2 | Функция активации №3 . . . . .                           | 11 |
|     | ЗАКЛЮЧЕНИЕ . . . . .                                     | 13 |
|     | ПРИЛОЖЕНИЕ А Листинг программы . . . . .                 | 14 |

## 1 Цель работы

Исследовать функционирование простейшей однослойной нейронной сети на базе одного нейрона с нелинейной функцией активации и выполнить её обучение по правилу Видроу–Хоффа.

## 2 Постановка задачи

Для варианта 2 требуется построить нейросетевую модель булевой функции четырёх переменных

$$F(x_1, x_2, x_3, x_4) = ((\overline{x_3} \vee x_4) \wedge \overline{x_1}) \vee x_2, \quad (1)$$

где  $x_1, x_2, x_3, x_4 \in \{0, 1\}$ .

Необходимо построить таблицу истинности, реализовать однослойную нейронную сеть, выполнить её обучение для функций активации №1 и №3, исследовать изменение суммарной квадратичной ошибки  $E(k)$ , определить выходные значения и весовые коэффициенты сети, а также подобрать минимальный набор обучающих векторов, после обучения на котором сеть правильно классифицирует все 16 входных комбинаций.

### 3 Теоретические сведения

Вход нейрона определяется выражением

$$net = w_0 + w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4, \quad (2)$$

где  $w_0$  — вес смещения, соответствующий постоянному входу  $x_0 = 1$ .

Для функции активации №1 используется пороговая функция

$$y(net) = \begin{cases} 1, & net \geq 0, \\ 0, & net < 0. \end{cases} \quad (3)$$

Коррекция весов выполняется по правилу

$$w_j^{(l+1)} = w_j^{(l)} + \eta \delta^{(l)} x_j^{(l)}, \quad (4)$$

где  $\eta = 0,3$ ,  $\delta = y_d - y$ .

Для функции активации №3 используется логистическая функция

$$y = f(net) = \frac{1}{1 + \exp(-net)}. \quad (5)$$

Её производная вычисляется как

$$f'(net) = f(net)(1 - f(net)), \quad (6)$$

а веса корректируются по выражению

$$w_j^{(l+1)} = w_j^{(l)} + \eta \delta^{(l)} f'(net) x_j^{(l)}. \quad (7)$$

При проверке модели непрерывный выход логистической функции переводится в бинарный класс по порогу 0,5.

Суммарная квадратичная ошибка за эпоху определяется выражением

$$E(k) = \sum_{l=1}^N (y_d^{(l)} - y^{(l)})^2. \quad (8)$$

## 4 Таблица истинности

Таблица 1 – Таблица истинности моделируемой булевой функции

| $x_1$ | $x_2$ | $x_3$ | $x_4$ | $F$ |
|-------|-------|-------|-------|-----|
| 0     | 0     | 0     | 0     | 1   |
| 0     | 0     | 0     | 1     | 1   |
| 0     | 0     | 1     | 0     | 0   |
| 0     | 0     | 1     | 1     | 1   |
| 0     | 1     | 0     | 0     | 1   |
| 0     | 1     | 0     | 1     | 1   |
| 0     | 1     | 1     | 0     | 1   |
| 0     | 1     | 1     | 1     | 1   |
| 1     | 0     | 0     | 0     | 0   |
| 1     | 0     | 0     | 1     | 0   |
| 1     | 0     | 1     | 0     | 0   |
| 1     | 0     | 1     | 1     | 0   |
| 1     | 1     | 0     | 0     | 1   |
| 1     | 1     | 0     | 1     | 1   |
| 1     | 1     | 1     | 0     | 1   |
| 1     | 1     | 1     | 1     | 1   |

## 5 Программная реализация

Программа реализована на языке Python. Комбинации входных переменных формируются с помощью функции **product**, а сочетания при поиске минимальной обучающей выборки — с помощью функции **combinations**. Номер функции активации и режим запуска передаются через **argparse**.

Доступны два режима работы программы:

- **train** — обучение на полной таблице истинности;
- **min** — последовательный перебор обучающих подмножеств возрастающей длины.

Полный текст программы приведён в приложении А.

## 6 Результаты обучения на полной выборке

### 6.1 Функция активации №1

Начальные веса равны нулю. Обучение завершилось за 12 эпох. На последней эпохе суммарная квадратичная ошибка стала равна нулю. Итоговый вектор весов:

$$W = (0; -0,9; 1,5; -0,3; 0,3). \quad (9)$$

В таблице 2 приведены результаты всех эпох обучения.

Таблица 2 – Параметры сети по эпохам для функции активации №1

| $k$ | Вектор весов $W$             | Выходной вектор $y$                              | $E(k)$ |
|-----|------------------------------|--------------------------------------------------|--------|
| 0   | (0.0; 0.0; 0.3; 0.0; 0.3)    | (1; 1; 1; 0; 1; 1; 1; 1; 1; 0; 0; 0; 0; 1; 1; 1) | 4      |
| 1   | (-0.3; -0.3; 0.6; 0.0; 0.3)  | (1; 1; 1; 0; 1; 1; 1; 1; 1; 1; 0; 0; 0; 1; 1; 1) | 5      |
| 2   | (0.0; -0.3; 0.9; 0.0; 0.3)   | (0; 1; 1; 0; 1; 1; 1; 1; 0; 1; 0; 0; 0; 1; 1; 1) | 5      |
| 3   | (0.0; -0.3; 1.2; 0.0; 0.3)   | (1; 1; 1; 0; 1; 1; 1; 1; 0; 1; 0; 0; 0; 1; 1; 1) | 4      |
| 4   | (-0.3; -0.6; 1.2; 0.0; 0.3)  | (1; 1; 1; 0; 1; 1; 1; 1; 0; 1; 0; 0; 1; 1; 1; 1) | 3      |
| 5   | (-0.3; -0.9; 1.2; 0.0; 0.3)  | (0; 1; 1; 0; 1; 1; 1; 1; 0; 1; 0; 0; 1; 1; 1; 1) | 4      |
| 6   | (0.0; -0.9; 1.2; 0.0; 0.6)   | (0; 1; 1; 0; 1; 1; 1; 1; 0; 0; 0; 0; 1; 1; 1; 1) | 3      |
| 7   | (0.0; -0.6; 1.5; 0.0; 0.6)   | (1; 1; 1; 1; 1; 1; 1; 1; 0; 0; 0; 0; 1; 1; 0; 1) | 2      |
| 8   | (-0.3; -0.6; 1.5; -0.3; 0.6) | (1; 1; 1; 1; 1; 1; 1; 1; 0; 0; 0; 0; 1; 1; 1; 1) | 1      |
| 9   | (-0.3; -0.9; 1.5; -0.3; 0.3) | (0; 1; 0; 1; 1; 1; 1; 1; 0; 1; 0; 0; 1; 1; 1; 1) | 2      |
| 10  | (0.0; -0.9; 1.5; -0.3; 0.3)  | (0; 1; 0; 1; 1; 1; 1; 1; 0; 0; 0; 0; 1; 1; 1; 1) | 1      |
| 11  | (0.0; -0.9; 1.5; -0.3; 0.3)  | (1; 1; 0; 1; 1; 1; 1; 1; 0; 0; 0; 0; 1; 1; 1; 1) | 0      |
|     |                              |                                                  |        |

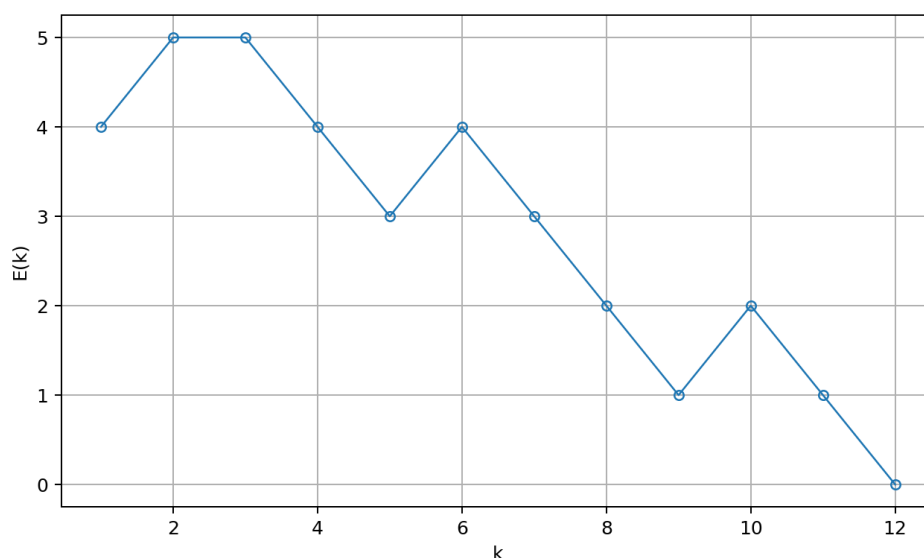


Рисунок 1 – Зависимость суммарной квадратичной ошибки от номера эпохи для функции активации №1

## 6.2 Функция активации №3

Для логистической функции активации установлен предел в 5000 эпох. За указанное число эпох условие  $E(k) < 0,01$  достигнуто не было. На последней эпохе получено

$$E(4999) = 0,0255314, \quad (10)$$

а итоговый вектор весов равен

$$W \approx (2,5308; -10,0775; 15,3251; -4,8175; 4,8660). \quad (11)$$

Несмотря на недостижение заданного порога суммарной ошибки, после преобразования выхода по порогу 0,5 все 16 комбинаций классифицируются правильно.

В таблице 3 приведены характерные эпохи обучения. Полная история всех 5000 эпох, включая вектор весов, выходной вектор и  $E(k)$ , сохранена в файле `teach_process_func_3.csv`, входящем в проект.

Таблица 3 – Параметры сети на выбранных эпохах для функции активации №3

| $k$  | Вектор весов $W$                        | Выходной вектор $y$                                                                                              | $E(k)$  |
|------|-----------------------------------------|------------------------------------------------------------------------------------------------------------------|---------|
| 0    | (0.175; -0.028; 0.264; 0.043; 0.117)    | (0.500; 0.509; 0.519; 0.508; 0.518; 0.554; 0.552; 0.602; 0.551; 0.564; 0.525; 0.519; 0.503; 0.545; 0.552; 0.606) | 3.76636 |
| 1    | (0.256; -0.090; 0.459; 0.040; 0.180)    | (0.544; 0.581; 0.570; 0.586; 0.620; 0.673; 0.650; 0.709; 0.572; 0.604; 0.548; 0.561; 0.577; 0.631; 0.616; 0.677) | 3.21409 |
| 2    | (0.292; -0.166; 0.618; 0.017; 0.217)    | (0.564; 0.615; 0.588; 0.617; 0.681; 0.738; 0.702; 0.761; 0.569; 0.612; 0.541; 0.565; 0.615; 0.675; 0.643; 0.708) | 2.92310 |
| 3    | (0.306; -0.246; 0.757; -0.014; 0.241)   | (0.572; 0.632; 0.590; 0.628; 0.721; 0.778; 0.732; 0.792; 0.555; 0.605; 0.519; 0.552; 0.637; 0.700; 0.655; 0.723) | 2.71334 |
| 4    | (0.311; -0.326; 0.882; -0.046; 0.260)   | (0.576; 0.641; 0.586; 0.629; 0.751; 0.806; 0.754; 0.812; 0.536; 0.593; 0.493; 0.532; 0.652; 0.716; 0.661; 0.731) | 2.53884 |
| 5    | (0.312; -0.403; 0.997; -0.078; 0.277)   | (0.577; 0.646; 0.579; 0.627; 0.774; 0.827; 0.770; 0.827; 0.516; 0.577; 0.465; 0.509; 0.663; 0.729; 0.663; 0.737) | 2.38693 |
| 6    | (0.312; -0.477; 1.105; -0.107; 0.292)   | (0.577; 0.650; 0.572; 0.624; 0.794; 0.845; 0.784; 0.839; 0.495; 0.561; 0.437; 0.487; 0.672; 0.740; 0.664; 0.741) | 2.25316 |
| 7    | (0.311; -0.546; 1.206; -0.134; 0.306)   | (0.577; 0.653; 0.564; 0.620; 0.811; 0.860; 0.796; 0.851; 0.476; 0.546; 0.412; 0.467; 0.681; 0.750; 0.666; 0.745) | 2.13504 |
| 99   | (0.709; -2.686; 4.235; -0.929; 1.050)   | (0.670; 0.855; 0.453; 0.689; 0.993; 0.998; 0.982; 0.994; 0.124; 0.291; 0.050; 0.132; 0.901; 0.963; 0.782; 0.915) | 0.61841 |
| 499  | (1.253; -4.964; 7.680; -2.166; 2.250)   | (0.778; 0.971; 0.289; 0.788; 1.000; 1.000; 0.999; 1.000; 0.024; 0.191; 0.003; 0.026; 0.981; 0.998; 0.856; 0.983) | 0.23783 |
| 999  | (1.601; -6.363; 9.774; -2.907; 2.976)   | (0.832; 0.990; 0.214; 0.840; 1.000; 1.000; 1.000; 1.000; 0.009; 0.145; 0.000; 0.009; 0.993; 1.000; 0.890; 0.994) | 0.13329 |
| 1999 | (1.994; -7.935; 12.122; -3.723; 3.780)  | (0.880; 0.997; 0.151; 0.885; 1.000; 1.000; 1.000; 1.000; 0.003; 0.104; 0.000; 0.003; 0.998; 1.000; 0.921; 0.998) | 0.06752 |
| 2999 | (2.232; -8.884; 13.540; -4.210; 4.262)  | (0.903; 0.998; 0.122; 0.907; 1.000; 1.000; 1.000; 1.000; 0.001; 0.084; 0.000; 0.001; 0.999; 1.000; 0.935; 0.999) | 0.04413 |
| 3999 | (2.401; -9.558; 14.548; -4.553; 4.603)  | (0.917; 0.999; 0.104; 0.920; 1.000; 1.000; 1.000; 1.000; 0.001; 0.072; 0.000; 0.001; 0.999; 1.000; 0.944; 0.999) | 0.03246 |
| 4999 | (2.531; -10.077; 15.325; -4.817; 4.866) | (0.926; 0.999; 0.092; 0.929; 1.000; 1.000; 1.000; 1.000; 0.001; 0.064; 0.000; 0.001; 1.000; 1.000; 0.951; 1.000) | 0.02553 |

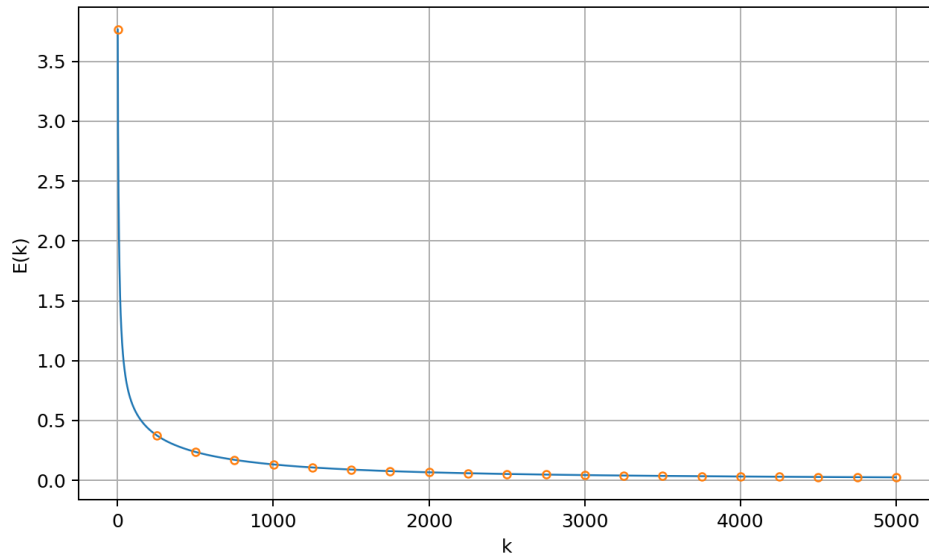


Рисунок 2 – Зависимость суммарной квадратичной ошибки от номера эпохи для функции активации №3

### 6.3 Итоговая проверка на полной таблице истинности

Таблица 4 – Результаты классификации после полного обучения

| $x_1$ | $x_2$ | $x_3$ | $x_4$ | $F$ | ФА №1 | Выход ФА №3 | Класс ФА №3 |
|-------|-------|-------|-------|-----|-------|-------------|-------------|
| 0     | 0     | 0     | 0     | 1   | 1     | 0.9263      | 1           |
| 0     | 0     | 0     | 1     | 1   | 1     | 0.9994      | 1           |
| 0     | 0     | 1     | 0     | 0   | 0     | 0.0922      | 0           |
| 0     | 0     | 1     | 1     | 1   | 1     | 0.9295      | 1           |
| 0     | 1     | 0     | 0     | 1   | 1     | 1.0000      | 1           |
| 0     | 1     | 0     | 1     | 1   | 1     | 1.0000      | 1           |
| 0     | 1     | 1     | 0     | 1   | 1     | 1.0000      | 1           |
| 0     | 1     | 1     | 1     | 1   | 1     | 1.0000      | 1           |
| 1     | 0     | 0     | 0     | 0   | 0     | 0.0005      | 0           |
| 1     | 0     | 0     | 1     | 0   | 0     | 0.0641      | 0           |
| 1     | 0     | 1     | 0     | 0   | 0     | 0.0000      | 0           |
| 1     | 0     | 1     | 1     | 0   | 0     | 0.0006      | 0           |
| 1     | 1     | 0     | 0     | 1   | 1     | 0.9996      | 1           |
| 1     | 1     | 0     | 1     | 1   | 1     | 1.0000      | 1           |
| 1     | 1     | 1     | 0     | 1   | 1     | 0.9508      | 1           |
| 1     | 1     | 1     | 1     | 1   | 1     | 0.9996      | 1           |
|       |       |       |       |     |       |             |             |

## 7 Поиск минимального набора обучающих векторов

В режиме `min` перебираются сочетания строк таблицы истинности, начиная с длины 1. Для каждого сочетания выполняется обучение с теми же параметрами, что и в режиме полного обучения. После завершения обучения полученная сеть один раз проверяется на всех 16 входных комбинациях. Первое сочетание, для которого итоговая классификация полной таблицы оказывается правильной, принимается за минимальное.

Следует учитывать, что значение  $E(k)$  в этом режиме вычисляется по выбранной обучающей выборке, поэтому его нельзя напрямую сравнивать с суммарной ошибкой режима полного обучения, где в эпохе участвуют все 16 векторов.

### 7.1 Функция активации №1

Минимальная обучающая выборка содержит 4 вектора. Первое найденное сочетание имеет индексы

$$(0; 3; 10; 14). \quad (12)$$

Обучение завершилось за 7 эпох с нулевой ошибкой на выбранной выборке. Итоговые веса:

$$W = (0; -0,6; 1,2; -0,3; 0,3). \quad (13)$$

Таблица 5 – Минимальная обучающая выборка для функции активации №1

| Индекс | $x_1$ | $x_2$ | $x_3$ | $x_4$ | $F$ |
|--------|-------|-------|-------|-------|-----|
| 0      | 0     | 0     | 0     | 0     | 1   |
| 3      | 0     | 0     | 1     | 1     | 1   |
| 10     | 1     | 0     | 1     | 0     | 0   |
| 14     | 1     | 1     | 1     | 0     | 1   |

После обучения на минимальной выборке полученная модель была проверена на всех 16 комбинациях входных переменных. Результаты проверки приведены в таблице.

Таблица 6 – Таблица истинности для модели с функцией активации №1, обученной на минимальной выборке

| $x_1$ | $x_2$ | $x_3$ | $x_4$ | $F$ | Выход сети |
|-------|-------|-------|-------|-----|------------|
| 0     | 0     | 0     | 0     | 1   | 1          |
| 0     | 0     | 0     | 1     | 1   | 1          |
| 0     | 0     | 1     | 0     | 0   | 0          |
| 0     | 0     | 1     | 1     | 1   | 1          |
| 0     | 1     | 0     | 0     | 1   | 1          |
| 0     | 1     | 0     | 1     | 1   | 1          |
| 0     | 1     | 1     | 0     | 1   | 1          |
| 0     | 1     | 1     | 1     | 1   | 1          |
| 1     | 0     | 0     | 0     | 0   | 0          |
| 1     | 0     | 0     | 1     | 0   | 0          |
| 1     | 0     | 1     | 0     | 0   | 0          |
| 1     | 0     | 1     | 1     | 0   | 0          |
| 1     | 1     | 0     | 0     | 1   | 1          |
| 1     | 1     | 0     | 1     | 1   | 1          |
| 1     | 1     | 1     | 0     | 1   | 1          |
| 1     | 1     | 1     | 1     | 1   | 1          |
|       |       |       |       |     |            |

## 7.2 Функция активации №3

Для логистической функции активации минимальная найденная выборка содержит 3 вектора с индексами

$$(5; 6; 10). \quad (14)$$

Обучение на этом наборе завершилось за 417 эпох. Суммарная квадратичная ошибка по трём обучающим векторам составила

$$E = 0,00999155, \quad (15)$$

а вектор весов равен

$$W \approx (0,2987; -2,4573; 2,7559; -0,4302; 0,7288). \quad (16)$$

Таблица 7 – Минимальная обучающая выборка для функции активации №3

| Индекс | $x_1$ | $x_2$ | $x_3$ | $x_4$ | $F$ |
|--------|-------|-------|-------|-------|-----|
| 5      | 0     | 1     | 0     | 1     | 1   |
| 6      | 0     | 1     | 1     | 0     | 1   |
| 10     | 1     | 0     | 1     | 0     | 0   |

После обучения на минимальной выборке модель была проверена на всех 16 комбинациях. Для логистической функции дополнительно приведён непрерывный выход сети и соответствующий ему бинарный класс.

Таблица 8 – Таблица истинности для модели с функцией активации №3, обученной на минимальной выборке

| $x_1$ | $x_2$ | $x_3$ | $x_4$ | $F$ | Выход сети | Класс |
|-------|-------|-------|-------|-----|------------|-------|
| 0     | 0     | 0     | 0     | 1   | 0,5741     | 1     |
| 0     | 0     | 0     | 1     | 1   | 0,7364     | 1     |
| 0     | 0     | 1     | 0     | 0   | 0,4672     | 0     |
| 0     | 0     | 1     | 1     | 1   | 0,6451     | 1     |
| 0     | 1     | 0     | 0     | 1   | 0,9550     | 1     |
| 0     | 1     | 0     | 1     | 1   | 0,9778     | 1     |
| 0     | 1     | 1     | 0     | 1   | 0,9324     | 1     |
| 0     | 1     | 1     | 1     | 1   | 0,9662     | 1     |
| 1     | 0     | 0     | 0     | 0   | 0,1035     | 0     |
| 1     | 0     | 0     | 1     | 0   | 0,1931     | 0     |
| 1     | 0     | 1     | 0     | 0   | 0,0699     | 0     |
| 1     | 0     | 1     | 1     | 0   | 0,1347     | 0     |
| 1     | 1     | 0     | 0     | 1   | 0,6451     | 1     |
| 1     | 1     | 0     | 1     | 1   | 0,7902     | 1     |
| 1     | 1     | 1     | 0     | 1   | 0,5417     | 1     |
| 1     | 1     | 1     | 1     | 1   | 0,7101     | 1     |
|       |       |       |       |     |            |       |

После обучения на трёх векторах проверка на полной таблице истинности показала правильный бинарный класс для всех 16 комбинаций.

## ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы была реализована однослойная нейронная сеть для моделирования булевой функции четырёх переменных. Для функции активации №1 при обучении на полной таблице истинности нулевая суммарная ошибка была достигнута за 12 эпох. Минимальная обучающая выборка для этой функции содержит 4 вектора.

Для логистической функции активации при полном обучении за 5000 эпох значение суммарной квадратичной ошибки уменьшилось до 0,0255314, но заданный программой порог 0,01 достигнут не был. При этом итоговая классификация всех 16 комбинаций оказалась правильной. В режиме поиска минимальной выборки найден набор из 3 векторов, после обучения на котором сеть также правильно классифицировала полную таблицу истинности.

Полученные результаты показывают различие между величиной непрерывной ошибки логистического выхода и результатом бинарной классификации по порогу 0,5, а также демонстрируют возможность обучения сети на сокращённом наборе образцов.

## ПРИЛОЖЕНИЕ А

### Листинг программы

Листинг A.1 – lab1.py

```
import argparse
import matplotlib.pyplot as plt
from math import exp
from itertools import product, combinations

TETTA = 0.3
MAX_EPOCH = 5000
X = []
F = []

def bull(x1, x2, x3, x4):
    return ((not x3) or x4) and (not x1) or x2

def build_table():
    X.clear()
    F.clear()
    for x1, x2, x3, x4 in product([0, 1], repeat=4):
        X.append([1, x1, x2, x3, x4])
        F.append(int(bull(x1, x2, x3, x4)))

def func(net, number):
    if number == 1:
        return 1 if net >= 0 else 0
    elif number == 3:
        return 1 / (1 + exp(-net))
    else:
        raise ValueError("Invalid function number")

def func_diff(net, number):
    if number == 3:
        y = func(net, number)
        return y * (1 - y)
    return 1

def calculate_net(W, x):
    net = 0
    for i in range(len(W)):
```

```

        net += W[i] * x[i]
    return net

def to_class(y, function_number):
    if function_number == 1:
        return y
    return 1 if y >= 0.5 else 0

def teach_model(function_number, train_indexes, verbose=False):
    W = [0.0, 0.0, 0.0, 0.0, 0.0]
    error_points = []
    for epoch in range(MAX_EPOCH):
        epoch_error = 0
        for index in train_indexes:
            x = X[index]
            target = F[index]
            net = calculate_net(W, x)
            y = func(net, function_number)
            error = target - y
            epoch_error += error ** 2
            if function_number == 1:
                for j in range(len(W)):
                    W[j] += TETTA * error * x[j]
            elif function_number == 3:
                y_diff = func_diff(net, function_number)
                for j in range(len(W)):
                    W[j] += TETTA * error * y_diff * x[j]
        error_points.append(epoch_error)
        if verbose:
            print("epoch =", epoch, "W =", W, "E =", epoch_error)
        if function_number == 1 and epoch_error == 0:
            break
        if function_number == 3 and epoch_error < 0.01:
            break
    return W, error_points, epoch + 1

def test_model(function_number, W, verbose=False):
    result = True
    for i in range(len(X)):
        net = calculate_net(W, X[i])
        y = func(net, function_number)
        prediction = to_class(y, function_number)

```

```

        if verbose:
            print(X[i][1:], "target =", F[i], "output =", y, "class"
                  "=", prediction)
        if prediction != F[i]:
            result = False
    return result

def full_training(function_number):
    indexes = range(len(X))
    W, error_points, epochs = teach_model(function_number, indexes,
                                           True)
    print()
    print("Epochs:", epochs)
    print("W:", W)
    print("E:", error_points[-1])
    print()
    print("Test:")
    result = test_model(function_number, W, True)
    print()
    print("Result:", result)
    plt.figure()
    plt.plot(range(1, len(error_points) + 1), error_points, marker=
             "o", markerfacecolor="none", markersize=4, linewidth=1)
    plt.xlabel("k")
    plt.ylabel("E(k)")
    plt.grid()
    plt.show()

def test_with_less_data(function_number):
    for taken in range(1, len(X) + 1):
        print()
        print("Testing length:", taken)
        comb = combinations(range(len(X)), taken)
        checked = 0
        for indexes in comb:
            checked += 1
            W, error_points, epochs = teach_model(function_number,
                                                    indexes)
            if test_model(function_number, W):
                print()
                print("Minimum length:", taken)
                print("Epochs:", epochs)

```

```

        print("Checked combinations:", checked)
        print("Indexes:", indexes)
        print("W:", W)
        print("Training error:", error_points[-1])
        print()
        print("Training vectors:")
        for index in indexes:
            print(X[index][1:], "->", F[index])
        print()
        print("Test on all 16 vectors:")
        test_model(function_number, W, True)
        return

    print("No suitable combinations with length", taken)
print("No suitable combination found")

def print_truth_table():
    print("x1 x2 x3 x4 | F")
    for i in range(len(X)):
        print(X[i][1], X[i][2], X[i][3], X[i][4], "|", F[i])

parser = argparse.ArgumentParser()
parser.add_argument("-f", "--func", type=int, choices=[1, 3],
                    required=True, help="Activation function: 1 or 3")
parser.add_argument("-m", "--mode", choices=["train", "min"],
                    required=True, help="train - full training, min - minimum
                    training set")
args = parser.parse_args()

build_table()
print_truth_table()

if args.mode == "train":
    full_training(args.func)
elif args.mode == "min":
    test_with_less_data(args.func)

```